

Hi! The Hazel editor was being weird to my MapFusion proof when exporting and reopening, so just in case, here are screenshots of what the MapFusion theorem proof looked like before I exported. I did induction on `xs`.

```

Induction on: xs

Case [] :
  assume xs == []

  == <map> (f, <map> (g, [])) == <map> (<compose> (f, g), [])
  apply function

  == case <map> (g, [])
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == <map> (<compose> (f, g), [])

  == case <map> (g, [])
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == case []
  | [] => []
  | hd::tl => (<compose> (f, g)(hd)::(<map+>)(<compose> (f, g), tl)):[ ]
  end

  == case <map> (g, [])
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == []

  == case case []
  | [] => []
  | hd::tl => (g(hd)::(<map+>)(g, tl)):[ ]
  end
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == []

  == case []
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == []

  == [] == []
  check equality

  == true

```

```

== true

Case h::t :
  assume xs == h::t
  assume map(f, map(g, t)) == map(compose(f, g), t)

  == <map> (f, <map> (g, h::t)) == <map> (<compose> (f, g), h::t)

  == <map> (f, <map> (g, h::t)) == case h::t
  | [] => []
  | hd::tl => (<compose> (f, g)(hd)::(<map+>)(<compose> (f, g), tl)):[ ]
  end

  == <map> (f, <map> (g, h::t)) == <compose> (f, g)(h)::<map> (<compose> (f, g), t):[ ]

  == <map> (f, <map> (g, h::t)) == <compose> (f, g)(h)::<map> (f, <map> (g, t)):[ ]

  == case <map> (g, h::t)
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == <compose> (f, g)(h)::<map> (f, <map> (g, t)):[ ]

  == case case h::t
  | [] => []
  | hd::tl => (g(hd)::(<map+>)(g, tl)):[ ]
  end
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == <compose> (f, g)(h)::<map> (f, <map> (g, t)):[ ]

  == case g(h)::<map> (g, t):[ ]
  | [] => []
  | hd::tl => (f(hd)::(<map+>)(f, tl)):[ ]
  end == <compose> (f, g)(h)::<map> (f, <map> (g, t)):[ ]

  == f(g(h))::<map> (f, <map> (g, t)):[ ] == <compose> (f, g)(h)::<map> (f, <map> (g, t)):[ ]

  == f(g(h))::<map> (f, <map> (g, t)):[ ] == <anon fun> (h)::<map> (f, <map> (g, t)):[ ]

  == f(g(h))::<map> (f, <map> (g, t)):[ ] == f(g(h))::<map> (f, <map> (g, t)):[ ]

  == true

```